Proof compression for the ZFC busy-beaver construction

A new four-row 215-state variant, checked local equivalence, context certificates, and limits of the 214-state search

Research packet for Vladimir Reshetnikov's ProveIt repository

21 October 2026

Abstract

The most economical way to certify nearby machines is to prove a small whole-tape simulation against the already certified primary K215 and reuse its calculus calibration. We construct a distinct 215-state machine, FM215, by changing four transition rows. Two immediate incoming-edge facts give its invariant; every parent transition is simulated by one or three new transitions. The generic transfer theorem and the exact-table, all-initial-tapes equivalence have been compiled in Lean 4.32.0 with only `propext` and `Quot.sound` as reported dependencies.

For shortening the parent proof itself, we identify a remaining Rocq certificate-symmetry port and construct a 234-role finite certificate for the parent's 4,750 internal decoder-prefix occurrences. The latter is an executable internal-decoder certificate, not yet a Lean or Rocq runtime replacement. A separate compiled Lean countermodel excludes a logic-only recovery step proposed for the large-matrix experiment, without deciding that experiment's full equiconsistency. A bounded search of 1,761,648 local replacement rows across 15 machines, together with a separate helper search, yields no verified 214-state machine. The packet includes exact tables, proofs, scripts, receipts, source pins and inherited notices.

Decisions and scope

1. **Use FM215 for a compact incremental correctness proof.** It has a simpler invariant than the existing conditional-marker CM215. It remains a 215-state machine: the removed logical role is reused as a physical selector, so no physical state is saved.
2. **Refactor the shared proof boundary before regenerating large certificates.** Positive simulation, one-way certificate symmetry, and context-indexed decoder sharing target actual repeated obligations.
3. **Keep the 214-state question open.** No 214-state candidate is offered as correct. The search limits and the logical obstruction are explicitly restricted results, not lower bounds.

Contents

Decisions and scope **1**

1 What is proved, and which machine is meant **3**

2 Audit of the existing formal proofs **3**

2.1 A concrete remaining simplification in Rocq 4

2.2 Use a verified parent as the proof boundary 5

3 A reusable positive simulation theorem **5**

4 FM215: the short correctness argument **6**

4.1 How the formal certificate binds the machine 7

4.2 Independent check of the existing CM215 route 8

5 A context certificate for the primary decoder **8**

5.1 The finite check and its whole-tape meaning 9

5.2 How to turn this into a smaller formal proof 10

6 A practical proof architecture **10**

7 An obstruction to the large-matrix logical recovery **11**

8 What the search toward 214 establishes **13**

8.1 Where to search next 14

9 Novelty, reproducibility and incorporation **14**

9.1 Files and commands 15

9.2 Recommended repository integration order 15

1 What is proved, and which machine is meant

All repository comparisons use the fixed commit

`b6c7087477e51c017003bd729e712c8969733223`.

Here "primary K215" means the native table at `candidate-k215/k215.native.tm`, whose exact hash appears in Section 4. The machine convention is one two-way infinite binary tape, initially all zero; every transition moves left or right; the halt label is separate from the working-state count. Initialization is part of the machine. FM215 has 430 working transition rows and 215 working states. Its native start, dispatcher and halt labels are $0, 84, 254$; its dense labels are $0, \ldots, 214$, with start 0, dispatcher 83 and halt 215. The supplied bijection specifies the renaming exactly.

Let $\mathrm{Ref}_{\mathrm{ZFC}}$ denote existence of a finite refutation in the ZFC calculus calibrated by the repository, and define $\mathrm{Con}(\mathrm{ZFC}) = \neg\,\mathrm{Ref}_{\mathrm{ZFC}}$. The inherited primary theorem and the new ordinary simulation argument compose as

$$\mathrm{Halts}(\mathrm{FM215}, \mathbf{0}) \longleftrightarrow \mathrm{Halts}(\mathrm{K215}, \mathbf{0}) \longleftrightarrow \mathrm{Ref}_{\mathrm{ZFC}}\,.$$

Negating this equivalence gives the requested

$$\mathrm{ZFC} \vdash \neg\,\mathrm{Halts}(\mathrm{FM215}, \mathbf{0}) \longleftrightarrow \mathrm{Con}(\mathrm{ZFC}).$$

The added argument consists of finite table facts and induction on runs and is formalizable in ZFC. This statement does not assume or establish Con(ZFC), and therefore does not assert unconditional nonhalting. As with the repository's documented verification boundary, an external proof-assistant theorem is distinguished from an explicitly encoded object-theory derivation. This packet does not emit the code of a ZFC or PA proof of the displayed statement.[1]

Claim	Evidence in this packet	Exact limitation
FM215 and primary K215 equihalt from state 0 on every tape	Compiled Lean theorem, exact literal-table binding audit, ordinary proof	Functional tapes; the adapter to the repository's zipper/dense endpoint is described but not compiled here.
FM215 has the same calibrated blank entry	Complete synchronized 95,714/95,840-step prefix with identical whole tape	Supporting check; unbounded equivalence follows from the Lean simulation.
Internal decoder has 234 context roles	468 checked branch rules and source-DAG expansion equality	Primitive first-row binding only; primitive runtime remains to be proved.
Missing low-binder vacuity is not logically recoverable	Ordinary countermodel and compiled Lean named logical interface	Excludes nonlogical seed bundles; no full-calculus inconsistency claim.
No successful 214-state reduction in tested families	Reproducible search scripts and counts	Bounded templates and overapproximate invariants; no machine lower bound.

2 Audit of the existing formal proofs

The inspected ProveIt revision is `b6c7087477e51c017003bd729e712c8969733223`. The repository already contains three proof-simplification checkpoints. Consequently, deriving a shared

AX-word interpreter, removing the reverse Lean source certificate, or replacing the original expanded dominated-rule derivation would repeat work already present in this revision.[1, 2]

The current Lean primary dependency closure has 185 project modules, compared with 204 at the original full-build checkpoint. The latest phase-three record freshly rebuilds nine affected modules and binds the 176 reused objects to their earlier checked sources, dependencies, objects and logs. Its two explicit audit clients produce ten reports containing only `propext`, `Classical.choice`, and `Quot.sound`. This is an incremental check of the complete dependency closure, not a fresh compilation of all 185 modules in this session. The original Rocq record reports 237 selected sources, four closed endpoint assumption reports and a recursive `coqchk`. Subsequent simplification records retain that baseline and identify the affected rebuilt modules.

The precise endpoint in Lean is

```
KEndpoint215.blank_halts_iff_zfc_refutation :
  BlankHalts machine215 <-> BProv ZFCax_s [] Form.fBot
```

The Rocq dense endpoint is `tm215_halts_iff_ZFC_inconsistent` in `k225-formal/rocq/rocq-extension/coq/215/CorrectDense215.v`. Both refer to the primary machine's literal dense table. The core composition is already short: the Lean endpoint is a direct application of the shared `KEndpoint` theorem, and Rocq rewrites the dense/sparse equivalence and applies its sparse endpoint. Shortening those final lines has little mathematical value. The substantial work is in the reusable simulation and calculus layers, and in the machine-specific certificates.[4]

Proof source	Original bytes	Current bytes
Lean dominated implication	689,857	35,185
Lean AX routine	202,222	7,563
Lean source data	653,569	456,548
Lean site-coverage facade	107,243	14,916
Lean register determinism	17,209	12,988
Rocq dominated implication	921,691	22,673

These are the repository's recorded source-byte comparisons, rather than measurements of the size of the resulting kernel proof terms. They also do not count every retained historical receipt as removed repository data.[10] The current Lean dominated-rule argument has a 61-lemma bootstrap, 81 reused conditional schemas, eight numerical dominance hypotheses, and a deduction theorem. The 629-node/735-instruction certificate remains the checked Rocq rendering; it is not the current Lean rendering.

The audit sources are the primary verification README, the simplification README, the phase-three README and its Lean record, and the actual endpoint files. Useful current source anchors are `KEndpoint215.lean:43` (the endpoint), `KSim.lean:317` (positive-step halting transfer), `RMOffsetBisim.lean:287` (one certificate gives equivalence), `L2AxiomsPerm.lean:100` (AX-word composition), and `DomRho8.lean:597` (the unrestricted dominated-row conclusion). All of these anchors refer to the pinned revision above.

2.1 A concrete remaining simplification in Rocq

The Lean source correspondence now uses a single forward certificate and the mathematical symmetry of an offset bisimulation. The current Rocq `Source215.v` still separately evaluates

```
certificate_forward : ocheck (ptree p215) (ptree dagPerm) Cert215 = true
certificate_backward :
  ocheck (ptree dagPerm) (ptree p215) (omirror Cert215) = true
```

with `vm_compute` (lines 8–13), and the same directory's `OffsetBisim230.v:167`, `ocheck_iff`, requires both results. Thus the same mathematical improvement remains available in the Rocq lane.

The appropriate addition is the generic theorem

```
ocheck_mirror t1 t2 C :
  ocheck t1 t2 C = true ->
  ocheck t2 t1 (omirror C) = true.
```

Its proof has five small ingredients. Lookup in `omirror C` is lookup in `C` followed by swapping the two paths and negating the integer offset. The identity $\delta(-z, i) = -\delta(z, i)$ turns each old count equation into its reverse. A simple completeness lemma for `counts_ok`, paired with the existing `counts_ok_sound`, avoids reasoning about the order of the duplicate register indices in its test list. These facts transfer a branch check, then an entry check, then the initial and universal checks. The initial offset is zero, so it is fixed by negation. This removes the second whole-certificate evaluation while preserving the exact theorem and every table/source input. This Rocq change is a proposed port; the new files in this packet do not claim to have compiled it.

2.2 Use a verified parent as the proof boundary

For a small transition-table edit, the most effective next simplification is often a direct equivalence proof with the already verified machine. The entire numerical-calculus calibration, proof enumeration, source bisimulation, site coverage and blank initializer can then remain behind one proved interface. The new obligation is a local simulation theorem about the changed transition graph.

This applies especially well to the new FM215 construction below. The existing conditional-marker alternative CM215 also has an ordinary all-tape equivalence with the primary machine, but its invariant needs an extra fact about state 156 and a two-step incoming path. FM215 uses two unconditional predecessor-marker facts, each established by the immediately preceding transition. Its additional correctness proof is therefore structurally simpler.

This comparison is about the additional proof after the primary endpoint has been proved. It is not a claim that changing the table removes the underlying set-theoretic calibration. Likewise, the rebalanced-address alternative's maximum source depth of 41 can reduce some certificate arithmetic, but a shorter address is not itself a shorter correctness argument. The composed allocation shares the primary source and seed, which makes it a conservative candidate for reusing the existing certificate pipeline, but it still needs its own physical runtime certificates unless an additional direct equivalence is established.

3 A reusable positive simulation theorem

Let $f : X \to X \cup \{\bot\}$ and $g : Y \to Y \cup \{\bot\}$ be deterministic partial transition functions. A configuration is terminal precisely when its transition is $\bot$. Let $R \subseteq X \times Y$ satisfy:

1. If $R(x, y)$ and $f(x) = x'$, then some y' satisfies $R(x', y')$ and $y \longrightarrow_g^k y'$ for some $k \geq 1$.
2. If $R(x, y)$ and $f(x) = \bot$, then the g-computation from y halts after finitely many steps.

Then $R(x, y)$ implies that f halts from x if and only if g halts from y. The relation need not be injective, surjective, or functional. Reachability invariants can simply be included in R.

For the forward implication, induct on the finite source computation, concatenating the finitely many target blocks. For the reverse implication, induct on the number t of successful target steps before termination. If the source is terminal, the conclusion is immediate. Otherwise its next step is simulated by a block of length $k \geq 1$. This block cannot extend past the target's terminal configuration: determinism and the absence of a transition there would contradict its certified successful completion. Hence $k \leq t$, and the induction hypothesis applies at the next related boundary with the strictly smaller bound $t - k$.

Positive progress is essential. If zero-length target blocks were allowed, a nonhalting source self-loop could be related to a terminal target forever. No uniform upper bound on block lengths is needed for halting equivalence. In FM215 there is nevertheless the useful exact bound $k \in \{1, 3\}$.

The supplied `PositiveSimulation.lean` is a generic extraction of the final argument in the repository's `KSim.halts_iff`. It uses arbitrary configuration types and deterministic partial transitions, so it can also serve machine-to-machine optimizations. It is not presented as a new abstract computability theorem. Its value is as a small reusable formal interface. The accompanying execution receipt records a successful Lean 4.32.0 compilation of this theorem. Its printed dependencies are exactly `propext` and `Quot.sound`.

4 FM215: the short correctness argument

Write T for the primary native K215 table and N for FM215. Define the state map

$$\phi(127) = 124, \qquad \phi(q) = q \quad (q \neq 127).$$

The two relevant old states have rows

	0	1
124	$0R62$	$1R138$
127	$0R14$	$1R138$.

Their entire incoming-edge census is $104/1 = 1R124$ and $110/0 = 0R127$. Hence, on every old computation from initial control zero and on every initial tape,

$$q = 124 \Longrightarrow u(h-1) = 1, \qquad q = 127 \Longrightarrow u(h-1) = 0,$$

where u denotes the current tape and h the current head position. Call this two-clause assertion $\mathcal{I}$. It holds initially because control zero is neither exceptional state, and it holds after *every* old transition: the two possible destination states force their marker through their unique incoming edge. In particular its preservation requires no hypothesis about the preceding tape.

FM215 changes exactly four rows:

row	primary K215	FM215
124/0	$0R62$	$0L127$
127/0	$0R14$	$0R8$
127/1	$1R138$	$1R109$
110/0	$0R127$	$0R124$.

The existing provider rows are $8/0 = 0R14$ and $109/0 = 0R62$. At simulation boundaries relate configurations with the same entire tape and head, old invariant $\mathcal{I}$, and new control $\phi(q)$. An ordinary old transition is simulated by one new transition. On the common read-one branch, both old exceptional states go to 138, and their new representative 124 does the same.

On an exceptional read-zero branch, let the predecessor marker be m. The new computation is

$$(124, h) \longrightarrow (127, h-1) \longrightarrow (x_m, h) \longrightarrow (a_m, h+1),$$

where $(x_0, a_0) = (8, 14)$ and $(x_1, a_1) = (109, 62)$. The three writes are respectively $0, m, 0$, each equal to the value already present. Thus the entire tape is unchanged. The invariant picks the correct old destination, and the head has moved right exactly once. All three internal controls have a transition, so no halt is hidden in the block. The state map fixes the terminal label 254.

The positive simulation theorem now proves halting equivalence on all binary initial tapes, with any initial head position, from control zero. If $z(t)$ counts the old read-zero visits to states 124 or 127 in a successful prefix of length t, the exact corresponding new duration is

$$t + 2z(t), \qquad t \le t + 2z(t) \le 3t.$$

This is a statement about arbitrary whole tapes, not a finite sample of register values and not an inference from the blank prefix.

4.1 How the formal certificate binds the machine

`FM215Data.lean` embeds both complete native tables and defines functional tapes $\mathbb{Z} \to \{0, 1\}$. The ambient label type is `Fin 512`, because the native tables use labels up to 302. This is an indexing device: exactly 215 labels have working rows, and every outgoing target is a working label or the separate terminal label 254. The finite ambient type does not assert a 512-state machine.

`FM215.lean`'s `incoming` theorem checks the two required incoming-row properties for every ambient state and both symbols. `outer` checks all unchanged and redirected rows under ϕ. Both are finite kernel decisions. The subsequent proofs establish invariant preservation, the complete three-step tape identity, and the one-or-three-step simulation. The endpoint is

```
FM215.all_tapes_halts_iff (h : Int) (t : Tape) :
  Halts (step old) (0,h,t) <-> Halts (step new) (0,h,t).
```

The actual file uses the corresponding structure notation and qualified `PositiveSimulation.Halts`. The blank instance is obtained by $h = 0$ and the constantly false tape.

All three supplied modules were compiled successfully in this session with the official Lean 4.32.0 release, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. The generic theorem and both FM215 endpoints report exactly `propext` and `Quot.sound`; there is no admitted theorem, extra assumption, or native evaluation oracle. The final compiler logs contain no warnings. `fm215-kernel-receipt.json` binds the checked sources, compiled objects and logs by SHA-256. The successful sequential run took about 26 seconds in this environment; this is a reproducibility datum rather than a comparative benchmark. The generic proof file has 99 lines, the FM215 proof file 172, and the table-and-semantics file 918, counting comments and blank lines. The first two files together are 10,375 bytes. `check_fm215.py` reproduces the compilation and the binding audit without any dependency on the ProveIt project build.

The independent `check_lean_bindings.py` reads every literal Lean row and compares it with the exact `.tm` input. It also checks the ambient bound, so a large numerical literal cannot silently wrap under `Fin`'s numeral instance. The primary native SHA-256 is

```
9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2
```

and the FM215 native SHA-256 is

```
3ece1233f11d33375425221c5bcedba941e073763912990eabf321197772dce5
```

The data audit is separate from the Lean compiler check. An exact-table theorem in functional-tape semantics is also separate from composing the repository's existing `KEndpoint215` theorem,

whose interface uses its own dense and zipper semantics. The outstanding imported bridge has two specific parts: identify the old native table with `KTable215.lean`'s `tm215` on working and terminal controls under the repository's renaming $254 \mapsto 230$, $301 \mapsto 102$, $302 \mapsto 120$; and relate functional tapes to `KZip`'s two finite lists, proving the one-step and terminal cases. The existing `KDenseBridge` already supplies the dense-to-zipper side. Alternatively one can adapt that generic bridge directly from Core's finite-support tapes to functional tapes, using the native dense relabelling. Then the new blank equivalence composes with `KEndpoint215`. An endpoint for the new dense table also uses FM215's supplied canonicalization map. These bridges must be explicit if a new end-to-end Lean consistency endpoint is claimed. No new blank bootstrap or calculus derivation is needed. Ordinary mathematical composition with the established primary calibration already gives the desired ZFC equivalence.

4.2 Independent check of the existing CM215 route

For comparison, `check_cm215.py` in this packet independently checks the existing six-row conditional-marker alternative against the exact primary bytes. Its invariant is

$$q = 52 \Rightarrow u(h-1) = 1, \quad (q = 176 \;\&\; u(h) = 0) \Rightarrow u(h-1) = 0, \quad q = 156 \Rightarrow u(h+1) = 1.$$

An old radius-two tape window determines the radius-one invariant after one L/R transition. Enumerating all 32 such windows per working control is therefore an exact finite proof-obligation check for arbitrary tapes. Of 6,880 state/window combinations, 6,840 satisfy the invariant. All preserve it and pass the whole-window block identity; 6,824 use one step and 16 use three. The checker also verifies the supplied native/dense bijection. It does not import the earlier symbolic runtime checker, the earlier blank trace, or the source calculus. The ordinary locality argument lifts these finite obligations to whole tapes because every read and write lies inside the certified window and exterior cells are untouched. The simpler FM215 invariant is the reason to prefer its new formal proof as the next compact endpoint.

5 A context certificate for the primary decoder

The next substantial shared-proof improvement is to replace repeated prefix-level decoder checks by a certificate over shared source nodes and the physical context that their implementation actually uses. It is not enough to hash-cons the source tree alone: the same subtree can have different physical entry controls, and some decoder paths inspect one or two preceding bits. A proof must retain those distinctions.

The exact `sites215.json` input has 4,751 primitive leaves and 4,750 internal-prefix occurrences. Its maximally shared syntactic DAG has 202 nodes: 179 internal nodes and 23 distinct primitive labels including their arguments. Pairing each internal node with its physical entry control gives 180 roles. We then refine a role by the last at most two bits of the sentinel-and-prefix string $1p$:

$$r = (v, q, \mathrm{suffix}_2(1p)).$$

The resulting certificate has **234 roles**. At the root the sentinel suffix is the one-bit string 1 and the physical control is 84.

Measured object	Count
Expanded internal-prefix occurrences	4,750
Shared internal source nodes	179
Internal source-node/control pairs	180
Pairs refined by one preceding bit	196
Pairs refined by two preceding bits	234
Internal branch rules in the two-bit certificate	371
Primitive-entry branch rules	97
Distinct primitive first-transition signatures	68

The larger number of roles after adding context is deliberate: source sharing and physical correctness require different equivalence relations. Two bits suffice for this certificate; no minimality assertion is made. Refining by three, four or five bits would yield 296, 388 or 523 roles. These counts are obtained from the actual literal table and source, not from an estimate based on source-tree depth.

5.1 The finite check and its whole-tape meaning

For an internal branch, instantiate the role's suffix immediately to the left of head position zero and put the branch bit at zero. Execute the literal table until the head first reaches position one. The certificate requires a positive number of steps, at most seven; every read and write must stay inside the explicitly represented suffix and current cell; every represented cell must be restored; and the final control must be the target role's control. The target source node must be the appropriate child and its suffix must be

$$\mathrm{suffix}_2(s\, b),$$

where s is the previous suffix and b the branch bit.

The validator also checks the DAG's acyclic child order, expands it to the exact pinned leaf list, checks the exact root role, rejects duplicate roles, and requires every supplied role to be reachable from the root. All two branches of every role are checked. For a primitive child the rule binds its operation and argument to the literal first transition from the role's control and branch bit.

Proposition 5.1 (Internal traversal)**.** *Suppose a full binary tape contains the sentinel and source address prefix expected at an internal decoder role, with the head at the next address bit. The internal rules of the supplied certificate take the machine to the appropriate next role, preserve the entire tape, and advance the head by one address cell in a positive finite number of transitions. Iterating these rules reaches the correct primitive entry for every source address.*

Proof. For one internal branch, all inspected cells are in the explicit local window. Agreement on that window makes the concrete full-tape run follow the same transitions as the finite check. The full exterior is neither read nor written. The check restores the window, so all tape cells are preserved; its suffix equation establishes the next role's precondition. Induct along the finite source address. The verified DAG child relation ensures that each branch selects the corresponding source child. At a leaf, the checked primitive-entry rule identifies its first transition. No claim about execution beyond that first primitive transition is used. □

Across the expanded internal occurrences, there are 4,749 internal-child edges: 4,355 take one transition, 303 take three, 90 take five, and one takes seven. The maximum backward read is two cells. Thus the certificate captures the real local decoder behavior, including its longer blocks.

The implementation is `checks/decoder_contexts.py`. The data and receipt are in `evidence/decoder_context_certificate.json` and `evidence/decoder_context_receipt.json`. This is an ordinary finite certificate with a stated soundness argument. No Lean or Rocq theorem for this new checker has yet been compiled.

5.2 How to turn this into a smaller formal proof

The proposed implementation sequence is concrete:

1. Define a finite source DAG and its expansion, and prove one generic equality between DAG traversal and tree traversal. Keep the present source semantics unchanged.
2. Define a role with source node, physical state and a suffix of at most two bits. Prove a generic local-frame lemma for functional tapes or the existing zipper interface.
3. Reflect each of the 371 internal and 97 primitive-entry rules into the kernel. Use the generic path-induction theorem to cover arbitrary address prefixes. This replaces repeated expansions of the same internal simulation facts by one reference to the role certificate.
4. Separately provide the existing quantified runtime contracts for the primitive entries, generalizing over the preserved address prefix and exterior as needed. The 68 first-transition signatures are useful grouping keys; they are not 68 already proved runtime contracts.
5. Compose these contracts at the shared site-coverage interface, retain the old endpoint as a regression target, and measure kernel term sizes and rebuild times before removing historical proof data.

The present certificate establishes that substantial decoder repetition can be shared soundly. It does not justify a numerical claim that the whole correctness proof will shrink by 4750/234, nor that the existing guarded runtime obligations disappear. Its benefit must ultimately be measured in the full build, not inferred from source counts alone.

6 A practical proof architecture

The repository already shares much of its calculus and compiler proof. The appropriate next architecture makes three interfaces explicit rather than moving all certificates into a new monolithic development.

Machine equivalence for small mutations. Use `PositiveSimulation.halts_iff` as a common interface for transition-table edits. A candidate provides its invariant, state/configuration relation, local positive blocks and terminal case. The endpoint then reuses the parent's proof. FM215 is a compiled example. For a Rocq port, the same induction on the remaining target halting time applies; it requires neither an infinite trace nor an independent reverse simulation.

The direct route for incorporating FM215 into the existing Lean tree is to identify the embedded native rows with the repository's K230 labels (including $254 \mapsto 230$, $301 \mapsto 102$, $302 \mapsto 120$), prove the functional-tape step relation equivalent to `KZip` or the core tape semantics, and compose `KEndpoint215`. The existing `KDenseBridge` already supplies the dense/zipper portion of the parent interface. The new dense machine then uses its supplied finite renaming. This adapter is the remaining mechanical integration task; it does not require redoing the calculus or the 95,714-step initialization.

Finite structural certificates for compiler output. Use an indexed source DAG and context roles for the decoding skeleton; retain quantified contracts for register primitives. Prove the checker sound once and evaluate only small, machine-specific propositions. Bind each theorem to a literal table and source representation. A file hash is an excellent audit key, but it is not a substitute for the literal data or an equality proved in the kernel.

A stable calibration interface. Keep the numerical-calculus/ZFC equivalence behind its named theorem. Expose explicit hypotheses for numerical guards and encoding side conditions. Do not repeat the already completed shared AX interpreter, Lean mirror-certificate removal, or guarded dominated-rule refactor. The remaining Rocq mirror theorem is the smallest immediate source-level change; the decoder certificate is the larger structural project. [2, 6, 7, 5]

For one or two alternatives, direct simulation is likely to be much less work than proving a general optimizing compiler. If many future machines will use arbitrary reallocation and reencoding, a verified certificate checker becomes more attractive: its initial proof cost is amortized over new tables. Candidate selection should therefore compare three quantities: physical state count, number of distinct proof contexts, and strength of the required tape invariant. A source tree with fewer nodes or shorter addresses does not automatically win on the latter two.

7 An obstruction to the large-matrix logical recovery

The existing large-matrix experiment removes the final comparison gates by emitting the seed bundle only at parameters $c > a + b$. Its report explicitly leaves recovery of the missing atomic-vacuity interface open. The following result closes one proposed route: that interface cannot be recovered from the retained logical rows alone. It does not decide whether the seed theory itself supplies a different consistency reduction.

Write $R(a, b)$ and $E(a, b)$ for membership and equality atoms, and $Q_i A$ for the named universal-quantifier constructor. Let $\mathcal{L}_>$ contain truth, modus ponens, theorem generalization, the unrestricted classical implicational basis, the rows $\rho_1, \ldots, \rho_6$ of `Code254.lean`, and the merged equality row

$$E(c, a) \to (E(c, b) \to Q_{r(a,b)} E(a, b)), \qquad r(a, b) = 4P(a, b) + 1.$$

Its membership-vacuity axioms are only

$$R(a, b) \to Q_c R(a, b) \qquad (c > a + b).$$

Allowing the unrestricted classical basis makes the obstruction stronger: the accepted dominated ρ_8 instances are a subfamily of it. No seed-K bundles or other nonlogical axioms are included in $\mathcal{L}_>$.

Theorem. The formula $R(2, 3) \to Q_1 R(2, 3)$ is not derivable in $\mathcal{L}_>$. In particular, the retained logical subsystem does not recover atomic vacuity at every distinct binder.

Proof. For any nonempty set D and irreflexive binary relation R_D, take worlds to be all sequences $e : \mathbb{N} \to D$. Interpret atoms by

$$e \models R(a, b) \iff R_D(e(a), e(b)), \qquad e \models E(a, b) \iff e(a) = e(b).$$

Interpret implication and falsity classically, and replace ordinary single-variable quantification by *tail quantification*:

$$e \models Q_i A \iff \forall f \, [(\forall k < i, \; f(k) = e(k)) \Longrightarrow f \models A].$$

The relations $e \sim_i f \iff \forall k < i,\ e(k) = f(k)$ are equivalence relations. They are nested, and

$$\sim_i \circ \sim_j = \sim_{\min(i,j)}, \qquad Q_i Q_j A \ \equiv\ Q_{\min(i,j)} A.$$

This identity is an identity of the semantics for every formula A; it is not a new inference rule.

All retained logical rows are valid. Normality of universal quantification proves ρ_2, and the displayed identity proves ρ_4. The row ρ_6, $(\exists_i Q_i A) \to A$, is valid because if an i-accessible world satisfies $Q_i A$, then the original world is among its i-accessible worlds. The row ρ_1, $\exists_i E(i,j)$, is valid: if $j < i$, assign coordinate i the fixed value $e(j)$; otherwise assign coordinates i, j a common value of D. Both changes preserve the prefix below i. Equality substitution proves ρ_3 and ρ_5. For the merged equality row, $r(a,b) > a, b$, so $r(a,b)$-accessible worlds preserve coordinates a, b; the two equality premises imply their equality there as well. The classical implicational axioms are tautologies. Global validity is closed under modus ponens and under every Q_i, as required by theorem generalization.

If $c > a + b$, then $a, b < c$, so c-accessible worlds preserve both arguments of $R(a,b)$. Thus all permitted atomic-vacuity rows are valid. The designated numerical falsity $R(0,0)$ is false everywhere, since R_D is irreflexive. The numerical truth alias $I(0,0) = 0$ is harmless: both sides evaluate to true. Equivalently, compose this interpretation with the repository's recursive numerical decoder; its implication and quantifier equations remain valid with these semantics.

Finally choose $D = \{0,1\}$ and $R_D(x,y) \iff x < y$. Let $e(2) = 0$ and $e(3) = 1$. Then $e \models R(2,3)$. There is a world $f \sim_1 e$ with $f(2) = f(3) = 0$, so $f \not\models R(2,3)$ and $e \not\models Q_1 R(2,3)$. Hence the proposed implication is not valid in a semantics validating every axiom and inference rule of $\mathcal{L}_>$, and therefore is not derivable. □

What this resolves and what it leaves open. Ordinary first-order matrix padding, or an unproved appeal to arbitrary variable renaming, cannot complete the missing logical bridge. A proof that transforms an allowed high-binder vacuity instance into the displayed low-binder instance must use additional nonlogical assumptions or a rule not present in $\mathcal{L}_>$. This result does not assert that the full large-matrix calculus has this countermodel: its seed bundles have not been shown valid in tail semantics. It neither proves nor disproves that full calculus's equiconsistency with ZFC, and gives no state-count lower bound.

Executable finite audit. `check_tail_quantifiers.py` checks the counterexample on the 16 assignments of coordinates $0, 1, 2, 3$, all indicated atomic rows with parameters in that range, and the universal algebraic laws used for quantifiers. It checks normality from the finite accessibility relations, not by sampling formulas, and checks ρ_6 and commutation for all 65,536 propositions on those 16 worlds. The finite audit is an independent sanity check of the displayed example and semantics.

Compiled Lean interface. `TailQuantifiers.lean` additionally formalizes the all-natural tail semantics, the validity of each displayed logical row, soundness of an inductive proof system, and the explicit non-derivability conclusion. It imports only `Init` and was compiled with Lean 4.32.0. Its logical interface is stronger than the one needed here: it admits every membership-vacuity binder larger than both atom indices, and every merged equality helper binder larger than both equality indices. The theorem `not_valid_missing_vacuity` has no axioms; the soundness and non-derivability theorems use only `propext`, `Classical.choice`, and `Quot.sound`. There are no `sorry` or `admit` declarations. The file treats named logical formulas as predicates on worlds; the extension through the repository's numerical decoder is the ordinary constructor argument above, not a newly compiled numeric-calculus bridge. Neither the Lean file nor the finite audit includes the seed-K theory bundles.

Pinned repository sources. This note concerns the pinned ProveIt revision, in particular its large-matrix formalism supplement[9], `BusyBeaver/ZFC/Code254.lean`, `BusyBeaver/ZFC/KCode222.lean`, and `BusyBeaver/ZFC/KThmK.lean`. The logical schemas above are their literal named-formula interpretations. This is a new obstruction proof; the large-matrix source experiment and the numerical calculus are inherited work.

8 What the search toward 214 establishes

The search uses the primary K215, thirteen pinned alternatives and FM215. It begins with a sound local invariant: for each state, collect reachable seven-cell windows centered at the head. Initially state zero admits all 128 windows, representing arbitrary initial tapes. Under each literal transition, shift the window and admit both values of the newly exposed exterior bit. Taking the least fixed point gives an overapproximation of the concrete reachable windows. This abstraction is inductive but loses long-range tape correlations.

The broadest completed family examines pairs of noninitial controls with identical other-symbol transitions and different selected-symbol transitions. For that selected symbol, the two sets of admitted seven-bit windows must be nonempty and disjoint. It deletes one of the pair, redirects every incoming edge to the survivor, and tries each write, direction and surviving working-state target for the differing row; halt is excluded as a replacement target. Both choices of survivor are tested. The symbolic comparison is required to match the parent's mapped endpoint, head and all inspected tape cells for every admitted seven-cell window, branching on previously unknown exterior cells.

Search family	Cases	Successful candidates
Full-window differing-row replacements	1,761,648	0
Adjacent-marker subfamily (included above)	710,480	0
Separate two-helper-row resynthesis	88,395	0

The broad family has 1,029 eligible control pairs. Its replacement count is $1029 \cdot 2 \cdot (2 \cdot 2 \cdot 214) =$ 1,761,648 across the 15 tables. The symbolic exploration limits are 32 transitions, 13 inspected cells and 200 symbolic nodes, with the initial seven cells and at most six new exterior cells. The adjacent-marker subfamily uses 415 pairs and an eight-cell inspection limit. These are nested counts, not independent searches to add together. An earlier 190,032-case read-zero/right-moving search is also a subset and is not counted again.

The separate helper search checks $415 \cdot 213 = 88{,}395$ pair/helper choices. It allows both rows of one helper to change, imposes the exact three-step selector effects, and preserves old helper uses through a two-step equality on every symbol admitted by the window invariant. Those restrictions define its scope. An exploration-limit exit is unresolved, not a counterexample. A mismatch on an abstract window can also be caused by the overapproximation; it does not by itself reject the candidate's actual blank-tape behavior.

The retained scripts and per-table receipts make the family and counts inspectable. No search ranges over all 214-state tables. In particular, source changes, global role reallocations, richer tape invariants and longer simulation blocks remain outside these families. The result is no verified candidate, not evidence of a 215-state lower bound.

One promising fold has a more informative failure than a bound exit. In the commands120 table, merge controls 78 and 210, redirect entries to 78, and replace 78/0 by $0R201$. On source leaf 25, address 1101001000 and command CD3, take all nine registers zero, one zero spare and gap zero. The parent returns to the correct successor address after 610 transitions; this 214-state trial returns after 594 transitions with the one at address-cell position 8 lost and a one

spuriously added at position 11. The supplied independent checker verifies the valid input frame, exact source leaf, both full output tapes and the literal trial. This refutes the proposed uniform primitive-runtime contract. It does not by itself decide the trial's blank-run equivalence. Its table is explicitly retained as rejected research under `machine/research/`.

8.1 Where to search next

FM215 isolates the unresolved resource question precisely: its logical role merger consumes the freed control as the predecessor selector. To save a physical state, that selector behavior must be implemented by an already live helper without breaking the helper's old uses, or several roles must be reallocated together.

The next useful search is therefore a multi-row synthesis problem. Fix a small neighborhood, allow two or more helpers to change, give each its reachable tape contexts, and ask for positive local simulation blocks. Use symbolic counterexamples to refine the contexts. For every apparent winner, produce a certificate accepted by the same generic simulation interface used here. An SMT or SAT solution is a proposal generator; the small kernel theorem should remain the acceptance criterion.

A second route changes the source or proof enumeration. It can make a larger physical difference, but it must carry a consistency-calibration argument before its state count is meaningful. Section 7 shows why one tempting removal of numerical comparison gates cannot be justified by logic-only vacuity recovery. It leaves theory-specific translations open. Startup shortening alone is also insufficient: changing the initial entry without preserving its complete calibrated tape creates a new global invariant obligation.

9 Novelty, reproducibility and incorporation

The four-row construction was compared with every `.tm` path in the pinned ZFC tree: 622 paths, with no missing or unparsable table. Of these, 321 are complete graph-reachable 215-state tables, representing 84 distinct rooted transition graphs. FM215 matches none, even allowing global left/right reflection. `machine/novelty_current_tree.json` records each current Git blob and SHA-256. This is novelty relative to that exact corpus; no claim about all unpublished machines or priority outside the repository is made.

The dense FM215 table has SHA-256

```
1b602d3b37887060ce2d992cf3036f540e8ccff5d37ab06c80f19f1edd38cbc6
```

Its 95,840-step blank prefix contains 63 stretched parent steps, giving $95{,}714 + 2 \cdot 63 = 95{,}840$. At that boundary the whole tape, not just a bounded observation window, equals the parent's tape: dispatcher 84, head -8, 95 one-cells and zeros elsewhere. The inherited interpretation is registers $[4, 0, 0, 0, 0, 0, 0, 0, 0]$, gap one, and forty unit spares. This agreement is checked by the independent sparse-tape simulator.

9.1 Files and commands

Location	Contents
`report.tex`, `sections/`	This report's complete editable source.
`report.pdf`	Compiled and visually inspected report.
`formal/`	Three FM215 Lean modules, exact native tables, compiler logs, receipt, binding checker, and independent CM215 comparison.
`machine/`	Native/dense FM215, renaming, builder, startup/local checks, pinned comparison tables, novelty and search scripts/receipts.
`checks/`, `evidence/`	Decoder context certificate and audit; bundle verification entry point and receipts.
`research/` `tail-quantifiers/`	Lean obstruction proof, finite audit, compiler log and exact source/compiler receipt.
`inputs/`, `provenance/`	Exact source inputs, inherited licenses/notices, and pinned upstream evidence references.

From the packet root, the following commands require no ProveIt checkout and no network access after the tools are installed:

```
python3 -B verify_all.py
python3 formal/check_fm215.py --lean /absolute/path/to/lean
lean research/tail-quantifiers/TailQuantifiers.lean
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The default verifier uses Python's standard library and rechecks the construction, exact literal bindings, local/startup simulations, decoder certificate, CM215 comparison, finite countermodel and receipt source hashes. It does not silently describe saved compiler logs as a fresh kernel run. The explicit Lean commands replay the kernel proofs and require Lean 4.32.0; no Mathlib project is required. The FM215 driver sets its local import path. Generated compiled objects are intentionally not part of the portable source packet.

The full novelty audit additionally needs the pinned ProveIt Git tree. The two search scripts use standard Node.js and are optional, longer reproductions; their precise commands are in the top-level README. The archive manifest binds the shipped files. Rebuilding logs or the PDF can change their hashes, so compare the manifest before regeneration.

9.2 Recommended repository integration order

First add FM215 and its standalone equivalence as a named alternative, with the explicit ordinary composition and the remaining Lean adapter identified. Then complete that adapter and, if desired, port the small positive-simulation proof to Rocq. Next port Rocq's mirror-certificate symmetry. Finally integrate the context certificate behind the existing site-coverage interface and benchmark the full dependency closure. This order yields a small reviewable theorem at each step and preserves the current certified parent as a stable comparison point.

The parent construction, numerical calculus, ZFC calibration, compiler ancestry and previously accepted alternatives remain inherited work. The packet's new contributions are the FM215 four-row transformation and checked equivalence, the decoder context certificate, the named logical obstruction and its proof, and the bounded search/audit work. Original notices and licenses are retained. The report does not claim a fresh full rebuild of the repository's existing Lean or Rocq development.

References

[1] ProveIt, K215 formal verification record, pinned revision above.

[2] ProveIt, Phase-three proof-simplification record.

[3] ProveIt, Primary K215 candidate and provenance.

[4] ProveIt, `KEndpoint215.lean`.

[5] ProveIt, `KSim.lean` and `RMOffsetBisim.lean`.

[6] ProveIt, Rocq `Source215.v`.

[7] ProveIt, Rocq `OffsetBisim230.v`.

[8] ProveIt, Conditional-marker alternative.

[9] ProveIt, Large-matrix formalism experiment.

[10] ProveIt, Proof-simplification checkpoints and source-byte comparisons.